

Recursive lambdas

ISO/IEC JTC1 SC22 WG21
P0839R0
Richard Smith
richard@metafoo.co.uk
2017-10-16

Motivation

Lambdas are a useful tool for local code refactoring. However, we sometimes want to use the lambda from within itself, either to permit direct recursion or to allow the closure to be registered as a continuation. This is surprisingly difficult to accomplish well in current C++.

Example:

```
void read(Socket sock, OutputBuffer buff) {
    sock.readsome([&] (Data data) {
        buff.append(data);
        sock.readsome(*current lambda*);
    }).get();
}
```

One natural attempt to reference a lambda from itself is to store it in a variable and capture that variable by reference:

```
auto on_read = [&] (Data data) {
    buff.append(data);
    sock.readsome(on_read);
};
```

However, this is not possible due to a semantic circularity: the type of the auto variable is not deduced until after the lambda-expression is processed, which means the lambda-expression cannot reference the variable.

Another natural approach is to use a `std::function`:

```
std::function on_read = [&] (Data data) {
    buff.append(data);
    sock.readsome(on_read);
};
```

This approach compiles, but typically introduces an abstraction penalty: the `std::function` may incur a memory allocation and the invocation of the lambda will typically require an indirect call.

For a zero-overhead solution, there is often no better approach than defining a local class type explicitly.

Proposal

This paper proposes allowing a lambda to refer to itself within its own definition. Specifically, a lambda will be permitted to provide a name prior to its parameter list (mirroring the syntax for a regular function), and within the body of the lambda, that name will be an lvalue denoting the **this* of the closure object.

```
auto visit_tree = [&] visit(Tree *node) -> void { // self parameter visit names lambda object
    if (node->left) visit(node->left); // and can be used to make recursive calls
    out << node->value;
    if (node->right) visit(node->right);
};
// the name visit is not in scope here
```

The name of the lambda can be used to refer to the lambda for any purpose, including making copies of the lambda or capturing it within a nested lambda.

```
sock.readsome([&sock, &buff] on_read(Data data) {
    buff.append(data);
    sock.readsome(on_read); // make a copy of the lambda
    // ...
}).get();
```

If the lambda does not specify an explicit return type, recursive calls will not be possible until the first return statement has been seen, as a consequence of the usual auto return type deduction rules applied to the lambda's `operator()` function.

```

auto bad_fib = [] fib(int n) {
    // error, return type for closure's operator() not yet deduced
    return fib(n-1) + fib(n-2);
};
auto less_bad_fib = [] fib(int n) {
    if (n == 0 || n == 1) return 1;
    return fib(n-1) + fib(n-2); // ok
};

```

Implicit captures

When a lambda has a *capture-default* ([&] or [=]), we do not know the semantic properties of the closure type until we reach the end of the lambda's body, because we do not know which local variables it captures. We propose to handle this by treating the closure type as an incomplete type within the body of the lambda if the lambda has a *capture-default*. In that case, the self parameter of the lambda can still have its call operator invoked and it can be captured by reference in nested lambdas, but it cannot be copied.

```

auto copy = [&] copysome() {
    // ok, captures self parameter of lambda 'copysome'
    // by reference
    sock1.readsome([&] on_read(Data data) {
        // ok, can call copysome even though type is incomplete
        sock2.writesome(std::ref(copysome));
    });
};

```

For uses where the lambda needs to be copied, the user code will need to explicitly specify a list of captures.

Proposed wording

Change in grammar in 8.1.5 [expr.prim.lambda]:

lambda-declarator:

identifier_{opt} (*parameter-declaration-clause*) *decl-specifier-seq_{opt}* *noexcept-specifier_{opt}* *attribute-specifier-seq_{opt}* *trailing-return-type_{opt}*

Add a new paragraph at the end of 8.1.5 [expr.prim.lambda]:

If the optional *identifier* within the *lambda-declarator* is present, it declares the lambda's *self parameter*, which is a variable whose declarative region is the *lambda-expression's compound-statement*. For a lambda with closure type *T*, the *self parameter* is a variable of type *cv T&* initialized with the closure object, where *cv* is empty if *mutable* appears within the *decl-specifier-seq* and is *const* otherwise. [*Example*:

```

auto fib = [] self(int n) {
    if (n < 2) return n;
    return self(n-1) + self(n-2);
};

```

— end example]

Add a new paragraph before 8.1.5.1 [expr.prim.lambda.closure] paragraph 10 ("The *lambda-expression's compound-statement* yields the *function-body* of the function call operator [...]":

The closure type is an incomplete type until the end of the *compound-statement* if the *lambda-expression* has a *lambda-capture* that includes a *capture-default*. Otherwise, the closure type becomes a complete type at the beginning of the *compound-statement*. The point of declaration of the function call operator or function call operator template for the closure type precedes the *compound-statement* ([basic.scope.class]). [*Example*:

```

void f(int n) {
    auto a = [&] g(int k) -> int {
        return k ? g(k - 1) + n : 0; // OK, lookup into incomplete closure type finds member operator(.)
    };
    auto b = [&] h(int k) {
        [h] { h(0); } (); // error: cannot capture parameter h with incomplete type
    };
}

```

```
—}.  
}.
```

— *end example*]

Change in 8.1.5.1 [expr.prim.lambda.closure] paragraph 6:

[...] The value returned by this conversion function is the address of a function *F* that, when invoked, has the same effect as invoking the closure type's function call operator **on an unspecified object of the closure type**.
[...]

Alternatives

- Allow an auto variable to be referenced in its initializer in this special case.
Pro: no new syntax required, avoids duplicating variable name as lambda self parameter name
Con: requires a variable declaration, adds a special case and extra complexity to the language rules
- Allow *this* to be used to refer to the enclosing closure object within a lambda.
Pro: does not require new syntax
Con: conflicts with existing use for denoting enclosing member function's *this* parameter, does not handle nested lambdas
- Permit arbitrary use of self parameter within a lambda with a *capture-default*, by giving it a dependent type and substituting the actual type after parsing.
Pro: allows full use of self parameter in lambda with *capture-default*
Con: adds complexity and creates cases where *typename* and *template* keywords would surprisingly be necessary
- Treat self parameter as a name for the closure type's operator *()* function, rather than as a synonym for the lambda object.
Pro: consistent rule for all lambdas, regardless of *capture-default*
Con: does not permit lambda to pass itself (by copy or reference), requires special-case rule to support use (mutual recursion) within nested lambdas